KUKA 逐点 1 mm 新手教程

适合你当前的环境：Windows、PyCharm、Python 3.12.1。

这是一套独立练习。你将复用已采集的 KUKA 数据，让数值逆运动学先工作，再比较神经网络初值是否能减少搜索时间。最后才生成新的测试目标，检验每个目标的位置误差是否都不超过 1 mm。

你今天只需完成第 1 ~ 5 课。不必重新采集数据，不必重新挑选基座和末端。

你的问题	这次的做法
关节预测误差大	对预测角度做数值修正，再用正向运动学测末端误差
是不是层数不够	先比较有、无神经网络初值，再决定是否训练更好的网络
要逐点 ≤1 mm	检查最大误差、全部目标通过数、失败和超时
PyCharm 操作不熟	每课写清在哪里输入、成功标志和下一步

当前目标限定为复制的 **KUKA LBR iiwa 14 R820 七轴仿真模型。**末端是已选定的 **link8_resp** 对象坐标系；还没有标定实际工具 TCP，也没有做碰撞检测。

本机实测：两种方法各完成 2000 个开发验证目标，全部位置 ≤1 mm，均无超时、越限或无解。

方法	位置通过	最大位置误差
numeric	2000 / 2000	0.977201 mm
hybrid	2000 / 2000	0.996103 mm

这是开发验证结果，尚未开展新的最终审计。教程制作日期：2026-10-18；完整记录见 **验证记录.json。**

第 1 课：先认识新的路线

逆运动学（IK）的输入是“希望末端到哪里、朝向哪里”，输出是七个关节角。正向运动学（FK）则从关节角计算末端位姿。

这次运行顺序是：目标位姿 → 初始关节角 → 数值 IK 修正 → 用场景 FK 复核 → 保存逐点结果。

方法	第一个初值	后备初值
numeric：数值基线	七个关节均为零	从训练集选近邻关节角，再用固定随机初值
hybrid：混合方法	已有神经网络的预测角度	与 numeric 相同的训练集近邻及随机初值

所以 numeric 是**没有神经网络的数值基线**，它仍会检索训练样本。两条路线使用相同目标、相同后备初值池、相同搜索预算。

你以前的“160”指一层中的 160 个神经元。原模型只有一个隐藏层，隐藏层用 ReLU，输出层为线性层；它不是 160 层。现在继续复用这个较弱模型，是为了看清数值修正的作用。

混合方法不一定比基线快。若没有改善，保留基线并分析初值，而不是先加到十层、八十层。

第 2 课：打开正确的文件夹

在哪里：PyCharm。

1. 点击菜单 **File > Open。**

2. 选择下面的新文件夹，点击打开；若问在哪个窗口打开，选 **New Window。**

```
D:\Swansea-杜亦冯\biyeshj\output\tutorials\kuka_1mm_beginner
```

3. 看左侧 Project 顶部：应当显示 **kuka_1mm_beginner。**

4. 展开它，应当看到 **00_check_environment.py** 至 **04_inspect_result.py**、**data**、**model**、**scenes** 和 **runs。**

文件或文件夹	用途
00_check_environment.py	检查 Python、依赖及复制文件
01_check_geometry.py	核对七关节链、坐标系和 FK 标签
02_run_experiment.py	运行 numeric、hybrid 或二者比较
03_make_audit.py	参数冻结后，生成新的 1000 个审计目标
04_inspect_result.py	用中文查看已经保存的实验结果
data / model / scenes	已复制的数据、旧模型和验证过的场景
runs	每次运行自动保存到新子文件夹

旧的 **kuka_data_collection** 和 **tiago_ik_beginner** 仍然保留。请不要把旧项目的 TIAGo 评估脚本拿来评估 KUKA。

第 3 课：选 Python 3.12.1

在哪里：PyCharm 设置。本机的新虚拟环境已经准备好，不需要再创建一个。

1. 按 **Ctrl + Alt + S** 打开 Settings。
2. 搜索 **Python Interpreter**，打开项目解释器页面。
3. 点击 **Add Interpreter > Add Local Interpreter**，或当前版本的添加按钮。
4. 选择 **Existing environment**；若窗口有 **New / Existing**，选 **Existing**。
5. 点击路径旁的 **…**，选择新文件夹中的解释器，再点 **OK / Apply**。

```
D:\Swansea-杜亦冯\biyeshj\output\tutorials\kuka_1mm_beginner\.venv\Scripts\python.exe
```

成功标志：右下角或解释器设置显示 Python 3.12，并且解释器路径包含 **kuka_1mm_beginner**。

如果是在另一台电脑解压练习包

虚拟环境不随 ZIP 搬运。在安装好 Python 3.12 和 CoppeliaSim 后，双击新文件夹里的 **setup_environment.cmd**；它会创建 **.venv** 并安装 **requirements.txt**。本机已有环境时也可以重复运行。

如果 PyCharm 建议 **pip install -e 本文件夹**，不需要执行。这里按脚本运行，只安装 **requirements.txt** 的依赖。

第 4 课：在 Terminal 检查环境

在哪里：PyCharm 下方的 Terminal。看到 **PS D:\...>** 就说明这里是 PowerShell。下面代码块里的命令可以在这里输入。

先输入这一行并按 Enter，把终端定位到新文件夹：

```
Set-Location -LiteralPath 'D:\Swansea-杜亦冯\biyeshj\output\tutorials\kuka_1mm_beginner'
```

再逐行运行：

```
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe 00_check_environment.py
```

成功标志：出现 Python 3.12.1、NumPy 2.5.4，以及下面这行：

```
[OK] Environment and packaged files are ready.
```

如果提示缺少包，运行：

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

不要复制提示符。例如 **(.venv) PS D:\...>** 不是命令。也不要在 PowerShell 直接输入 **import numpy**；Python 语句应写进 **.py** 文件再运行。本教程的命令均已替你调用 Python。

第 5 课：检查模型几何

在哪里：同一个 Terminal。

```
.\.venv\Scripts\python.exe 01_check_geometry.py
```

脚本会自动启动一个隐藏的 CoppeliaSim，使用复制的场景和独立端口。完成后自动关闭这个后台进程。

你不需要打开可见的 CoppeliaSim 窗口，不需要点 Play，也不需要再次选择场景树对象。后台程序仍会运行；只是这次启动和退出由 Python 脚本管理。

检查内容包括：末端是否属于配置的基座、七个关节的顺序和限位是否匹配、50 条已有标签是否能由场景 FK 重现、四元数正负号是否正确处理。

成功标志：

```
[OK] Geometry and FK checks passed.
```

结果保存到自动新建的 **runs/geometry_check_001** 等目录。你会看到 **max_label_position_difference_mm**；这里的数值表示“标签与模型是否一致”，不是神经网络或者 IK 已达到的精度。

这一步使用真实关节标签检查 FK。后面的 IK 实验不会把待测目标的真实关节角作为初值。

如果本课失败，先停止后续实验，查看错误和 **logs**。第 14 课有排查方法。

第 6 课：先跑数值基线

在哪里：Terminal。

```
.\.venv\Scripts\python.exe 02_run_experiment.py --mode numeric
```

默认从开发验证集选择 100 个目标。每个目标先试零位初值，再按需要尝试训练集近邻和固定随机初值。所有角度必须在七个关节的限位以内。

你会看到 **Isolated CoppeliaSim ready**，然后看到 **Evaluated 10/100** 等进度。第一次启动后台程序需要稍等，先不要连续重复点击运行。

结束时会打印 **Saved:**。默认自动创建 **runs/numeric_validation_001**，下次则是 **_002**；无需每次自己想新名字。

需要区分的状态	含义
Process finished with exit code 0	程序完成了，结果已经写出
position_pass_count 等于目标数	所有目标的几何位置都通过阈值
有失败或超时	本次未达到全部实验要求，必须保留这些目标

程序正常结束不等于全部目标已经达到 1 mm。下一课教你直接读结果。

脚本只在复制的仿真场景中计算关节角；此练习不连接实体机械臂。

第 7 课：读懂结果文件

在哪里：Terminal。这一行会自动读取最近一次完成的实验，不会重新求解：

```
.\.venv\Scripts\python.exe 04_inspect_result.py
```

也可以在 PyCharm 左侧展开刚生成的 **runs** 子目录，双击 **summary.json** 或 **numeric_targets.csv**。

指标或文件	怎样理解
position_pass_count	在全部目标中，有多少位置误差通过
position_mm.max	所有返回的合法候选中，最大位置误差；还要同时检查无候选数
position_mm.mean / p95	平均值 / 95% 分位数，不能替代最大值
no_bounded_candidate_count	没有返回合法关节角的目标数，仍计为失败
budget_exceeded_count	超过每目标时间预算的数量
*_targets.csv	每一目标的误差、耗时、通过标志
*_details.json	每一次尝试及初值修正情况
parameters.json	参数、目标行号、版本和文件哈希

例如，99/100 通过，即使平均误差很小，也不满足“每个目标 ≤1 mm”。最大误差只针对有结果的候选；若另有一个目标无解，不能用这个最大值声称全通过。

第 8 课：再比较神经网络初值

在哪里：Terminal。

```
.\.venv\Scripts\python.exe 02_run_experiment.py --mode compare
.\.venv\Scripts\python.exe 04_inspect_result.py
```

这次在同一批 100 个目标上运行 numeric 和 hybrid，并交替两者的执行顺序。结果自动保存到新的 **compare_validation_...** 目录。

hybrid 先用旧神经网络预测七个角度，再做数值修正。若预测超限，会先裁剪为合法初值，并在结果中记录；裁剪后的初值不等于已经求解成功。

按这个顺序比较

1. 两种方法的位置通过数、无合法候选数和超时数。

2. 最大位置误差及朝向误差。

3. 每目标总耗时的平均值和 P95。

4. 使用的初值数量；查看后备初值是否经常承担主要作用。

总耗时包含初值检索、神经网络预测、远程通信和重试；不能只比较网络的一次前向传播。

若 hybrid 通过但比 numeric 慢，说明精度主要由修正器保证，当前神经网络尚没有明显加速收益。这也是有价值的实验结论。

本机这一次 2000 目标实验的平均总耗时约为 numeric 5.91 ms、hybrid 5.69 ms；差距很小，单次测量不足以证明显著加速。更换硬件或后台负载后，耗时会改变。

第 9 课：1 mm 究竟怎么算

对每个目标，取求解出的关节角，在复制场景中重新执行 FK，得到末端位置。位置误差按三维欧氏距离计算：

$$位置误差(mm) = 1000 \times \sqrt{dx^2 + dy^2 + dz^2}$$

坐标输入单位为米。三个方向分别不超过 1 mm，并不能确保三维距离不超过 1 mm。你的验收要求是每个测试目标的三维距离都 ≤1 mm，边界等号算通过。

检查	本教程默认值或规则
用户的位置目标	每点三维距离 ≤1 mm
数值求解内部目标	0.1 mm，保留收敛余量
朝向辅助阈值	最短三维旋转角 ≤0.5°，是暂定辅助条件
内部朝向目标	0.1°
关节限位	七关节全部满足硬限位
时间预算	默认每目标 1000 ms，超时另计并不能算预算内通过

朝向比较会把四元数 **q** 和 **-q** 视为同一个旋转。位置通过和位置、朝向、限位、时间全部通过会分别报告。

七轴机械臂可能有多个正确关节解。只要 FK 位姿、限位和所需条件通过，与采集时的真实角度不同也可以算正确，不能只按关节标签误差否定它。

仿真误差很小，说明这个模型中的目标得到了重现；实体机械臂还会受到标定、控制和工具误差影响。本教程只验收当前仿真端点。

第 10 课：扩大开发验证

100 个目标能让你熟悉流程。准备好后，在同一个开发验证集上评估全部 2000 个目标：

```
.\.venv\Scripts\python.exe 02_run_experiment.py --mode compare --samples 2000
.\.venv\Scripts\python.exe 04_inspect_result.py
```

先保存完整结果，再分析失败。对开发验证集可以调试，但不要把删掉困难目标后的结果称作 100% 成功。

可调参数	作用与代价
--max-iterations	每个初值允许的迭代预算；提高后可能更慢
--max-starts	包括首个初值在内的总初值上限
--time-budget-ms	每个目标的总时间预算
--seed	控制目标抽样和随机初值；比较时保持一致

要增大重试预算，先在开发集测试，例如：

```
.\.venv\Scripts\python.exe 02_run_experiment.py --max-starts 32 --samples 100
```

这行默认仍是 compare。不要同时改许多参数，否则难以判断哪项改变有用。提高预算可能提高通过率，也可能让速度变差；结果应一起报告。

当前脚本的时间预算在迭代块之间检查，一次正在执行的块可能稍晚返回。超时会如实记录，不能声称具有硬实时保证。

第 11 课：分清开发集和最终审计集

已有 10000 条数据沿用旧模型的划分：训练 7000、开发验证 2000、历史测试 1000。

训练行用于原模型训练和后备初值检索。验证行用于本教程开发、调试和选择参数。历史测试结果之前已经看过，所以这 1000 条不能再作为未见过的最终证据。

最终审计要在参数冻结后生成。今天初次练习不用运行下一课。

计划的新审计集包含 1000 个由场景 FK 生成的可达目标：

子集	数量与生成方法
常规随机	500，关节空间均匀抽样
接近限位	250，把 1 ~ 3 个关节放在限位内侧附近
近伸直	250，把第 2、4、6 关节约束在零位 ±3°

近伸直子集还没有逐点计算雅可比来证明奇异性，不能称它是已验证的奇异目标集。

审计 CSV 只保存目标位姿。用于证明可达的真实关节角保存在另一个 witness 文件里，求解器不会读取它们作为答案或初值。

第 12 课：冻结后再做最终审计

仅当开发阶段完成、参数决定不再修改时运行。随包的完整实测结果是 **runs/compare_validation_008**。下面以它为例；若你重新选择自己的结果，请替换为该目录。

```
.\.venv\Scripts\python.exe 03_make_audit.py --freeze-run runs/compare_validation_008
```

冻结记录会保存实验参数、随机种子、源结果记录以及模型、数据、场景和求解代码的哈希。它会生成 **data/audit_1000.csv** 及相关元数据。

然后用冻结时的相同参数运行，例如冻结结果用了全部默认参数时：

```
.\.venv\Scripts\python.exe 02_run_experiment.py --split audit
.\.venv\Scripts\python.exe 04_inspect_result.py
```

若冻结时修改过 **--max-starts** 等参数，审计命令也必须带上相同的值。脚本会拒绝参数、种子、核心文件或代码与冻结记录不一致的审计。

审计始终评估全部 1000 个目标，**--samples** 不能把它缩减为容易的一小部分。位置失败、无合法候选和超时都必须保留。

若审计失败，报告失败并回到开发阶段。修改方案之后，旧审计集就属于已知目标，不能继续反复调试它再声称“未见目标全部通过”。

即使本次审计全部通过，结论也只覆盖这一批有限仿真目标；它不证明整个工作空间都满足 1 mm。

第 13 课：不想用 Terminal 时

在哪里：PyCharm 左侧 Project。

1. 找到想运行的 **.py** 文件，例如 **00_check_environment.py**。

2. 在这个文件名上点击鼠标右键。

3. 选择 **Run '00_check_environment'**。

4. 看下方 Run 第一行：最后应当是你选中的脚本路径。

不要只看编辑器当前打开了什么文件。你以前打开“查看 npz”时，顶部运行配置仍是 **01_warmup_numpy**；点击顶部绿色三角形就会继续运行旧脚本。

给运行配置添加参数

打开 **Run > Edit Configurations**，确认 Script path 是本项目的 **02_run_experiment.py**，Interpreter 是本项目 **.venv**，Working directory 是本项目文件夹。

在 **Parameters** 中填写：

```
--mode compare --samples 100
```

不要把整条 PowerShell 命令填进 Parameters，只放脚本参数。

这次默认输出会自动递增目录名。只有手动指定已经存在的 **--out** 才会拒绝覆盖；换一个新名字或删掉这个参数即可保留旧结果继续运行。

第 14 课：遇到问题先看这里

现象或提示	应该做什么
import 无法识别 / from 不支持	当前是 PowerShell；运行教程提供的 python.exe 命令
No module named ...	确认新项目解释器，再用它执行 pip install -r requirements.txt
找不到脚本或 requirements.txt	先执行第 4 课的 Set-Location
CoppeliaSim executable not found	查看实际安装位置，用 --sim-exe 指向 coppeliaSim.exe
后台启动后没有可见窗口	本教程预期如此；看终端进度和结果目录
找不到对象 / FK 几何检查失败	检查是否修改了 scenes、model_config 或数据；先恢复配套副本
输出目录已存在	不指定 --out，使用自动递增目录
程序结束，但 position 未全部通过	这是有效失败结果；查看逐点 CSV，不要删困难目标
审计提示冻结信息不一致	使用被冻结的代码、模型、种子和参数；不能绕过校验

如果安装目录不同，可以先用下面的方式检查环境；其他需要模拟器的脚本也支持 **--sim-exe**：

```
.\.venv\Scripts\python.exe 00_check_environment.py --sim-exe 'C:\实际安装目录\coppeliaSim.exe'
```

这里的“实际安装目录”是示例占位，必须替换。你本机默认安装位置已配置好，正常不用添加这项。

后台连接报错时，保留完整报错和该次 **logs**。先运行 **00**、**01**，确认不是环境或几何问题，再分析求解器。

第 15 课：怎样继续研究

第一阶段的任务是让数值基线工作，确认误差计算和逐点验收可信。第二阶段才研究神经网络初值能否提高速度和稳定性。

值得模仿的工作	你可以学习什么
Lu、Zou、Jiang，2022	神经网络预测后接数值修正，以末端重建误差评价
CycleIK，2023	用 FK 位姿损失训练网络，并处理冗余关节的选择偏好
Vu 等，2023	在同类 KUKA 七轴上学习冗余参数，再结合解析 IK

Lu 等的工作使用六轴 xArm6；其中的高数值精度不能直接当作你这台七轴 KUKA 已达到的结果。本练习采用的是官方 simIK 阻尼最小二乘求解，不是对该论文分区网络和 Newton–Raphson 实现的逐行复现。

CycleIK 提供了“让网络学习末端几何”的思路。后续可以比较小网络、较深网络、FK 损失及冗余约束，但数据划分和评估目标要保持一致。

建议的毕业设计比较表至少保留：方法、全部目标数、位置通过数、最大/P95/平均位置误差、朝向误差、限位失败、超时数、平均/P95总耗时。

先把这些结果做扎实，再考虑更大数据集、真正的奇异性分层测试、碰撞约束和实体 TCP 标定。

参考资料与今日清单

参考来源均链接到论文原文或官方文档：

- Lu 等：基于神经网络和数值方法的逆运动学，Sensors 2022
- CycleIK 论文原文
- Vu 等：用深度学习增强解析逆运动学
- CoppeliaSim 官方运动学插件说明
- CoppeliaSim 官方 simIK API

今天按这个顺序做

1. 打开 **kuka_1mm_beginner**。
2. 选择它自己的 **.venv\Scripts\python.exe**。
3. 在 Terminal 定位到新文件夹。
4. 跑 **00_check_environment.py**，看到环境检查通过。
5. 跑 **01_check_geometry.py**，看到几何检查通过。
6. 跑 **02_run_experiment.py --mode numeric**。
7. 跑 **04_inspect_result.py**，读通过数与最大误差。
8. 跑默认 compare，与 neural 初值修正路线比较。

第一次练习先到这里。**03_make_audit.py** 是开发结束后的步骤，不是今天必须完成的安装检查。

所有 PowerShell 命令都可以从本 Markdown 文件复制。PDF 用于逐课阅读；结果 CSV 用于逐点分析，NPZ 用于保存模型和数组。